

```

from datetime import datetime

class CarnavalAmbato:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self):
        self.eventos = []

    Tabnine | Edit | Test | Explain | Document | Ask
    def agregar_evento(self, evento):
        self.eventos.append(evento)

    Tabnine | Edit | Test | Explain | Document | Ask
    def listar_eventos(self):
        for evento in self.eventos:
            print(evento)
            print()

```

```

class Evento:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, titulo, fecha_inicio, fecha_fin, organizador, lugar):
        self.titulo = titulo
        self.fecha_inicio = fecha_inicio
        self.fecha_fin = fecha_fin
        self.organizador = organizador
        self.lugar = lugar
        self.inscripciones = []

    Tabnine | Edit | Test | Explain | Document | Ask
    def agregar_inscripcion(self, inscripcion):
        self.inscripciones.append(inscripcion)

    Tabnine | Edit | Test | Explain | Document | Ask
    def __str__(self):
        return (f"Evento: {self.titulo}\nFecha de Inicio: {self.fecha_inicio.strftime('%d-%m-%Y')}\n"
                f"Fecha de Fin: {self.fecha_fin.strftime('%d-%m-%Y')}\nLugar: {self.lugar}\n"
                f"Organizador:\n{self.organizador}\nNúmero de Inscritos: {len(self.inscripciones)}")

class Inscripcion:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, participante, evento):
        self.participante = participante
        self.evento = evento

    Tabnine | Edit | Test | Explain | Document | Ask
    def __str__(self):
        return f"Inscripción de {self.participante.get_nombre_completo()} en el evento '{self.evento.titulo}'"

```

```
class Organizador:
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def __init__(self, cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto, lugar_de_trabajo):
    self.cedula = cedula
    self.nombres = nombres
    self.apellidos = apellidos
    self.direccion = direccion
    self.correo_electronico = correo_electronico
    self.telefono_contacto = telefono_contacto
    self.lugar_de_trabajo = lugar_de_trabajo
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def __str__(self):
    return f"Cédula: {self.cedula}\nNombre: {self.get_nombre_completo()}\nDirección: {self.direccion}\n"
        f"Email: {self.correo_electronico}\nTeléfono: {self.telefono_contacto}\nLugar de trabajo: {self.lugar_de_trabajo}"
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def get_nombre_completo(self):
    return f"{self.nombres} {self.apellidos}"
```

```
class Participante:
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def __init__(self, cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto):
    self.cedula = cedula
    self.nombres = nombres
    self.apellidos = apellidos
    self.direccion = direccion
    self.correo_electronico = correo_electronico
    self.telefono_contacto = telefono_contacto
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def get_nombre_completo(self):
    return f"{self.nombres} {self.apellidos}"
```

Tabnine | Edit | Test | Explain | Document | Ask

```
def __str__(self):
    return f"Cédula: {self.cedula}\nNombre: {self.get_nombre_completo()}\nDirección: {self.direccion}\n"
        f"Email: {self.correo_electronico}\nTeléfono: {self.telefono_contacto}"
```

```
class Factory:
```

Tabnine | Edit | Test | Explain | Document | Ask

```
@staticmethod
```

```
def crear_organizador(cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto, lugar_de_trabajo):
    return Organizador(cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto, lugar_de_trabajo)
```

Tabnine | Edit | Test | Explain | Document | Ask

```
@staticmethod
```

```
def crear_participante(cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto):
    return Participante(cedula, nombres, apellidos, direccion, correo_electronico, telefono_contacto)
```

```
def main():
    # Datos del evento y participantes
    organizador = Factory.crear_organizador("1723494430", "Ximena", "Carcelén", "Quito", "ximena@gmail.com", "0998238344",
                                           "Dirección de Cultura del Municipio de Quito")
    evento = Evento("Exposición de Pintura", datetime(2024, 2, 18), datetime(2024, 2, 23), organizador, "Casa de la Cultura")

    participante1 = Factory.crear_participante("1893948587", "Jesús Mateo", "Urbina", "Ambato", "jesus@gmail.com", "0998238344")
    participante2 = Factory.crear_participante("1822403844", "María Magdalena", "Quishpe", "Quito", "maria@gmail.com", "0995244382")

    inscripcion1 = Inscripcion(participante1, evento)
    inscripcion2 = Inscripcion(participante2, evento)

    evento.agregar_inscripcion(inscripcion1)
    evento.agregar_inscripcion(inscripcion2)

    carnaval = CarnavalAmbato()
    carnaval.agregar_evento(evento)
    carnaval.listar_eventos()

if __name__ == "__main__":
    main()

# Implementación de patrones de diseño:
# 1. Singleton: Podrías implementar el patrón Singleton en la clase CarnavalAmbato para asegurarte de que solo haya una
# instancia de CarnavalAmbato.
# No es necesario en este caso de clase

# 2. Factory: Podrías usar el patrón Factory para crear instancias de Evento, Inscripcion, Organizador y Participante.
# Necesario e implementado

# 3. Observer: Podrías implementar el patrón Observer para notificar a los participantes cuando se agregue un nuevo evento o inscripción.
# Muy poco útil en este caso de practica
```

asddadasadsadsa